

Informe Laboratorio 3

Sección 4

Estrella Bustos Aillapan
e-mail: estrella.bustos@mail.udp.cl

Octubre de 2025

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	2
2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio	3
2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión	4
2.3. Genera el hash de la contraseña desde la consola del navegador	5
2.4. Intercepta el tráfico login con BurpSuite	6
2.5. Realiza el intento de login por medio del hash	6
2.6. Identifica las políticas de privacidad o seguridad	8
2.7. Comente 4 conclusiones sobre la seguridad del sitio escogido	9

1. Descripción de actividades

Su objetivo será auditar la implementación de algoritmos hash aplicados a contraseñas en páginas web desde el lado del cliente, así como evaluar la efectividad de estas medidas contra ataques de tipo Pass the Hash (PtH). Para llevar a cabo esta auditoría, deberá registrarse en un sitio web y crear una cuenta, ingresando una contraseña específica para realizar las pruebas.

Al concluir la tarea, es importante que modifique su contraseña por una diferente para garantizar su seguridad.

Dado que la cantidad de sitios chilenos que utilizan hash es limitada, se permite realizar esta tarea en cualquier sitio web a nivel mundial. En este sentido, realice las siguientes actividades:

- Identificación del algoritmo de hash utilizado para las contraseñas al momento del registro en el sitio.
- Identificación del algoritmo de hash utilizado para las contraseñas al momento de iniciar sesión.
- Generación del hash de la contraseña desde la consola del navegador, partiendo de la contraseña en texto plano.
- Interceptación del tráfico de login utilizando BurpSuite desde su equipo.
- Realización de un intento de login modificando la contraseña por una incorrecta haciendo uso del hash obtenido en el punto anterior. Puede interceptar el tráfico y modificar el hash por el correcto o hacer uso del servicio repeater de BurpSuite.
- Descripción de las políticas de privacidad o seguridad relacionadas con las contraseñas, incluyendo un enlace a las mismas.
- Cuatro conclusiones sobre la seguridad o vulnerabilidad de la implementación observada.

2. Desarrollo de actividades según criterio de rúbrica

Antes de iniciar las pruebas se definieron las condiciones experimentales y las herramientas necesarias para garantizar reproducibilidad y trazabilidad en el análisis. Se detallan a continuación los elementos principales del entorno empleado.

1. **Navegadores:** Se utilizó Mozilla Firefox para inspección del DOM y depuración, y Chromium como navegador de pruebas conectado a Burp Suite por su compatibilidad con el proxy.
2. **Interceptación y manipulación:** Burp Suite Community Edition (Intercept, Repeater) para capturar, editar y reinyectar peticiones HTTP/HTTPS.

3. **Generación de hashes en cliente:** CryptoJS cargado desde CDN en la consola del navegador para reproducir exactamente el cálculo MD5 que realiza la página.

2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio

Durante el registro en *free-hack.com* se creó una cuenta de prueba con la contraseña LabCripto3. Al inspeccionar la petición de registro se observó que la contraseña no se transmitía en texto plano, sino mediante campos denominados `password_md5` y `passwordconfirm_md5`, lo que indica el uso de MD5 en el cliente.

Para documentar y verificar este comportamiento se realizaron dos pruebas complementarias: cálculo del digest desde la consola del navegador (replicando el proceso cliente) y comprobación independiente con un generador en línea.

```
timezoneoffset: "1"
dst: "2"
options[adminemail]: "1"
options[showemail]: "1"
agree: "1"
s: ""
securitytoken: "guest"
do: "addmember"
url: "https://free-hack.com/"
password_md5: "dd1d59ccf7fdf2e922de43259ca6a2b3"
passwordconfirm_md5: "dd1d59ccf7fdf2e922de43259ca6a2b3"
day: ""
month: ""
```

Figura 1: Cálculo del MD5 en la consola mediante CryptoJS.

Use this generator to create an MD5 hash of a string:

LabCripto3

Generate →

Your String	LabCripto3
MD5 Hash	dd1d59ccf7fdf2e922de43259ca6a2b3 Copy

Figura 2: Verificación externa del MD5.

La coincidencia exacta entre el digest generado en consola y el valor enviado en la petición confirma que el sitio efectúa MD5 en el lado cliente y no aplica salting visible en ese punto.

2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión

Al revisar el código fuente del formulario de inicio de sesión se identificaron campos ocultos nombrados `vb_login_md5password` y `vb_login_md5password_utf`. El script asociado al evento `onsubmit` prepara estos campos con el digest MD5 antes de serializar el formulario, indicando que el proceso de autenticación repite la misma lógica que el registro y que el servidor espera recibir el hash calculado por el navegador.

```

Benutzername und dein Kennwort in ...ieren <Schaltfläche, um
ein neues Benutzerkonto anzulegen." accesskey="s">
  whitespace
<input id="cb_cookieuser_navbar" class="cb_cookieuser_navbar"
type="checkbox" name="cookieuser" value="1" accesskey="c"
tabindex="103">
</div>
</div>
</fieldset>
<input type="hidden" name="s" value="">
<input type="hidden" name="securitytoken"
value="1729841665-9c31791fd2772e02d1e08efbacf2a9d9b9473424">
<input type="hidden" name="do" value="login">
<input type="" name="vb_login_md5password">
<input type="hidden" name="vb_login_md5password_utf">
</form>
▶ <script type="text/javascript"> ... </script>
</li>
</ul>

```

Figura 3: Fragmento del código cliente que muestra la preparación de campos MD5 en el formulario de login.

Esta implementación implica que la comprobación del secreto se realiza, en la práctica, comparando el digest enviado con el almacenado o esperado por el servidor, sin un challenge dinámico que impida la reutilización directa del digest.

2.3. Genera el hash de la contraseña desde la consola del navegador

Para reproducir el proceso de hashing de forma controlada se inyectó CryptoJS y se ejecutó el cálculo MD5 de la contraseña de prueba. El digest obtenido coincidió exactamente con el observado en las solicitudes, lo que demuestra la replicabilidad del valor por cualquier actor con acceso al entorno del navegador.

```

> const script = document.createElement('script');
script.src = 'https://cdnjs.cloudflare.com/ajax/libs/crypto-js/3.1.9-1/crypto-js.js';
document.head.appendChild(script);
▶ <script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/3.1.9-1/crypto-js.js">
> var password = "LabCripto3";
var hash = CryptoJS.MD5(password).toString();
console.log(hash);
dd1d59ccf7fdf2e922de43259ca6a2b3

```

Figura 4: Captura adicional que muestra el digest MD5 generado en consola (refuerzo visual).

Dicha replicabilidad facilita la captura y posterior reutilización del digest en escenarios donde un atacante obtenga acceso al valor calculado (por ejemplo, mediante un plugin malicioso, un XSS o registros inseguros).

2.4. Intercepta el tráfico login con BurpSuite

Con Chromium configurado para usar Burp Suite como proxy, se interceptó la petición de autenticación (POST /login.php?do=login). En el cuerpo de la petición se observó la presencia del digest MD5 en campos duplicados, así como parámetros asociados a la gestión de sesión.

```

1 POST /login.php?do=login HTTP/2
2 Host: free-hack.com
3 Cookie: bb_sessionhash=01900edb30141986536a44533f42d7a5; bb_lastvisit=1729855568; bb_lastactivity=0
4 Content-Length: 227
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="126"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept-Language: en-US
10 Upgrade-Insecure-Requests: 1
11 Origin: https://free-hack.com
12 Content-Type: application/x-www-form-urlencoded
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://free-hack.com/
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 vb_login_username=luccasniccolas&vb_login_password=&vb_login_password_hint=Kennwort&s=&securitytoken=quest&do=login&vb_login_md5password=
dd1d59ccf7fd2e922de43259ca6a2b3&vb_login_md5password_utf=dd1d59ccf7fd2e922de43259ca6a2b3

```

Figura 5: Petición de login interceptada por Burp Suite que muestra los campos con el digest MD5.

Aunque la comunicación se realiza sobre HTTPS, la funcionalidad investigada demuestra que el servidor valida la autenticación en base al digest recibido; esto deja abierta la posibilidad de reutilización del digest si un adversario logra obtenerlo.

2.5. Realiza el intento de login por medio del hash

Para evaluar la posibilidad de un ataque *Pass-the-Hash (PtH)*, se modificó la solicitud interceptada en Burp Suite. En primer lugar, se alteró el último carácter del *digest* MD5 con el fin de provocar un fallo de autenticación. A continuación se reenvió la petición con el hash incorrecto para comprobar la respuesta del servidor.

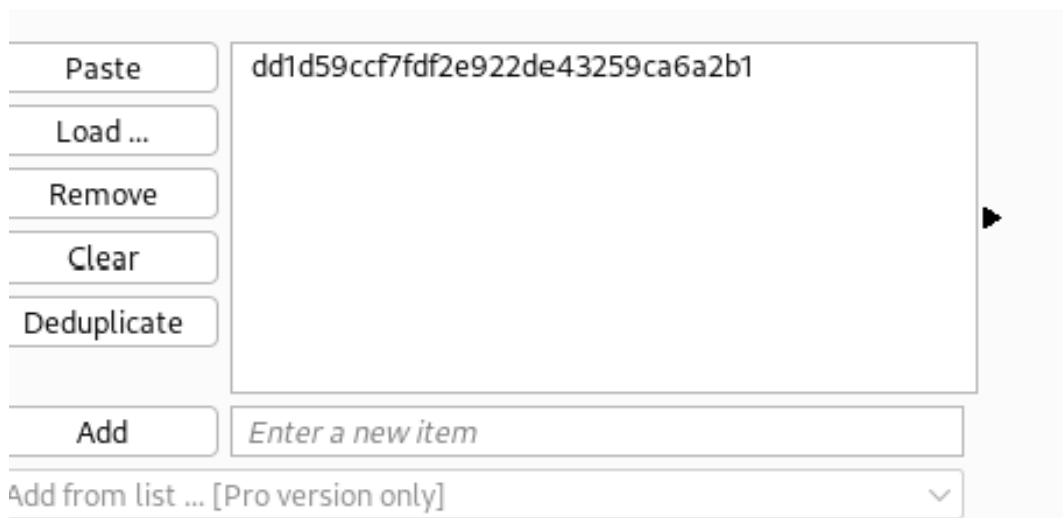


Figura 6: Modificación del hash MD5 para forzar un fallo de autenticación.

Al enviar la petición con el hash incorrecto, Burp Suite mostró que el servidor respondió presentando nuevamente el formulario de inicio de sesión, lo que indica que la autenticación había fallado. Esta respuesta refleja que el servidor rechazó el digest enviado al no coincidir con el esperado.

```
<!--
<script type="text/javascript" src="clientscript/vbulletin_md5.js?v=425">
</script>
<form id="navbar_loginform" action="login.php?do=login" method="post" onsubmit="md5Hash(vb_login_password, vb_login_md5password, vb_login_md5password_utf, 0)">
  <fieldset id="logindetails" class="logindetails">
    <div>
      <div>
        
        <input type="text" class="textbox default-value" name="vb_login_username" id="navbar_username" size="10" accesskey="u" tabindex="101" value="Benutzername" />
        
        <input type="password" class="textbox" tabindex="102" name="vb_login_password" id="navbar_password" size="10" />
        <input type="text" class="textbox default-value" tabindex="102" name="vb_login_password_hint" id="navbar_password_hint" size="10" value="Kennwort" style="display:none;" />
        <input type="submit" class="loginbutton" tabindex="104" value="Anmelden" title="Gib zur Anmeldung deinen Benutzernamen und dein Kennwort in die dafür vorgesehenen Textfelder ein oder klicke auf die 'Registrieren'-Schaltfläche, um ein neues Benutzerkonto anzulegen." accesskey="s" />
        <input type="checkbox" name="cookieuser" value="1" id="cb_cookieuser_navbar" class="cb_cookieuser_navbar" accesskey="c" tabindex="103" />
      </div>
    </div>
  </fieldset>
-->
```

Figura 7: Respuesta del servidor ante el intento de autenticación con hash incorrecto (autenticación fallida).

Posteriormente, se sustituyó el valor del parámetro por el hash MD5 correcto previamente obtenido ('dd1d59ccf7fdf2e922de43259ca6a2b3') y se reenvió la solicitud. Tras liberar la interceptación en Burp Suite, el servidor aceptó las credenciales y permitió el acceso a la cuenta asociada.

```
<body>
<div class="standard_error">
  <form class="block vbform" method="post" action="https://free-hack.com/" name="postvarform">
    <h2 class="blockhead">
      Weiterleitung . . .
    </h2>
    <div class="blockbody formcontrols">
      <p class="blockrow restore">
        Danke f&uuml;r deine Anmeldung, luccasniccolas.
      </p>
    </div>
    <div class="blockfoot actionbuttons redirect_button">
      <div class="group" id="redirect_button">
        <a href="https://free-hack.com/" class="textcontrol">
          Falls dein Browser dich nicht automatisch weiterleitet, klicke bitte hier.
        </a>
      </div>
    </div>
  </form>
```

Figura 8: Digest MD5 correcto utilizado en la autenticación exitosa.

Este comportamiento demuestra que el servidor valida directamente el valor del *digest* recibido sin aplicar un esquema de desafío-respuesta ni verificaciones adicionales sobre la contraseña en texto claro. Por tanto, el sistema es vulnerable a ataques *Pass-the-Hash*: un atacante que obtenga el hash y conozca el nombre de usuario puede autenticarse sin conocer la contraseña original.

2.6. Identifica las políticas de privacidad o seguridad

Durante la auditoría del sitio *free-hack.com*, se evaluaron tanto las prácticas de seguridad implementadas como la disponibilidad de políticas formales de privacidad y protección de datos. A continuación, se detallan los principales hallazgos obtenidos:

Protección de contraseñas: El sitio emplea el algoritmo MD5 para generar el hash de las contraseñas antes de enviarlas al servidor. Este procedimiento resulta obsoleto, ya que MD5 presenta vulnerabilidades conocidas frente a ataques de fuerza bruta, uso de tablas precalculadas (*rainbow tables*) y colisiones. Además, no se observó la aplicación de técnicas de *salting* en el proceso, lo que implica que contraseñas iguales producirán el mismo hash, incrementando el riesgo en caso de filtración. En la actualidad, se recomienda el uso de funciones de derivación de claves más robustas como *bcrypt*, *Argon2* o *PBKDF2*, que incorporan mecanismos de sal y coste computacional para mejorar la seguridad.

Medidas de seguridad en la autenticación: Se observó que la página no implementa medidas adicionales de protección en el proceso de inicio de sesión. En particular, no se detectaron mecanismos de limitación de intentos fallidos (*Rate Limiting*), lo cual facilita los ataques de fuerza bruta. Tampoco se emplean controles anti-automatización como *CAPTCHAs*, ni métodos de autenticación multifactor (MFA), los cuales podrían mitigar intentos no autorizados. Asimismo, la ausencia de un esquema de desafío-respuesta permite que el servidor acepte el hash como única credencial válida, dejando expuesto el sistema a ataques

del tipo *Pass-the-Hash*.

Políticas de privacidad y transparencia: No se encontró en el sitio una política de privacidad visible ni información sobre el tratamiento de los datos personales de los usuarios. La falta de documentación clara respecto a la recolección, almacenamiento y uso de la información reduce la transparencia y dificulta conocer el cumplimiento de normativas básicas de protección de datos. Esta ausencia también evidencia debilidades en la gestión de seguridad, ya que no se comunica cómo se resguardan las credenciales ni qué medidas técnicas se aplican para prevenir fugas o accesos no autorizados.

En conclusión, las prácticas observadas —uso de MD5 sin salting, hashing del lado del cliente, falta de controles de mitigación y ausencia de políticas claras— reflejan un nivel de seguridad insuficiente frente a estándares modernos de protección de credenciales y privacidad de los usuarios.

2.7. Comente 4 conclusiones sobre la seguridad del sitio escogido

El análisis realizado revela una serie de deficiencias en la gestión de credenciales que afectan la postura de seguridad del sitio. En primer lugar, el uso de MD5 para procesar contraseñas en el cliente se considera obsoleto: MD5 es vulnerable a ataques de colisión y su rapidez lo hace inadecuado para proteger secretos frente a intentos masivos de criptoanálisis. La falta de *salting* agrava esta debilidad, ya que contraseñas idénticas generan hashes idénticos, facilitando ataques por diccionario o la identificación de contraseñas reutilizadas entre cuentas.

En segundo lugar, la arquitectura que confía en el hash calculado por el cliente convierte al digest en una “credencial” válida por sí misma; en la práctica, esto habilita ataques del tipo *Pass-the-Hash* si un atacante consigue capturar el digest (por ejemplo, mediante la interceptación de una petición en un entorno comprometido, un plugin malicioso del navegador o un registro inseguro en logs). Dicha vulnerabilidad no se mitiga solamente con TLS, porque TLS protege el canal en tránsito, pero no impide que el valor sea obtenido en el extremo del cliente o del servidor si existen compromisos.

En tercer lugar, la ausencia de medidas de defensa complementarias (rate limiting, CAPTCHA, bloqueo progresivo, MFA) indica una postura de seguridad reducida que facilita la automatización de ataques. Estas contramedidas son especialmente relevantes cuando la función de hashing empleada es débil.

En cuarto lugar, la falta de una política de privacidad clara y de prácticas documentadas acerca del manejo de credenciales y registros limita la transparencia y la capacidad de auditoría del sitio. Publicar y aplicar políticas que describan el tratamiento de datos y las prácticas de almacenamiento es una parte esencial para generar confianza y facilitar auditorías externas.

En resumen, la implementación observada puede proporcionar una ilusión de protección superficial, pero no satisface las prácticas contemporáneas de seguridad para la gestión de contraseñas. Es recomendable migrar a funciones de derivación de claves robustas y de coste configurable (por ejemplo, `bcrypt` o `Argon2`), implementar *salting* por usuario, y desplazar la lógica de verificación al servidor en lugar de confiar en un digest calculado en el cliente. Además, se deben incorporar controles de acceso adicionales (MFA, rate limiting, bloqueo progresivo) y documentar las políticas de privacidad y seguridad para mitigar riesgos operativos y legales.